Lab 4

OOP trong kiểm thử & Mô hình Page Object Model (POM)

Hình thức nộp: Đường dẫn repository GitHub (public) — refactor trên repo của Bài 1–3

---

1. Mục tiêu

Sau bài tập này, sinh viên có thể:

- Nhận ra vấn đề của code script-based và giải thích được vì sao cần refactor.
- Áp dụng đóng gói & kế thừa để thiết kế Page Object.
- Refactor một bộ test có sẵn sang POM mà KHÔNG làm đổi hành vi.
- Viết code sạch: đặt tên tốt, hàm ngắn, tuân thủ DRY.
- Thẩm định được kết quả refactor do AI tạo ra.

2. Yêu cầu

Phần A — Cấu trúc & BasePage (20%)

- Tạo hai thư mục `Pages/` và `Tests/`; namespace khớp với thư mục.
- Viết `BasePage` (abstract) chứa: `driver`, `WebDriverWait`, và hàm `Tim(By)` tìm phần tử KÈM chờ.
- Mọi Page class phải kế thừa BasePage — **không** tự khai báo lại driver/wait.

Phần B — LoginPage & InventoryPage (25%)

- Locator để `private readonly By`; hành vi nghiệp vụ để `public`.
- LoginPage có tối thiểu: `DangNhap(tk, mk)` (trả về `InventoryPage`) và `LayThongBaoLoi()`.
- InventoryPage có tối thiểu: `DaHienThi()`, `SapXep(value)` (trả về `this`), `LayDanhSachGia()`.
- Tên hành vi phải trả lời "người dùng LÀM ĐƯỢC GÌ", không phải "trang có thẻ nào".

Phần C — Refactor test Bài 2–3 (30%)

- Chuyển toàn bộ ca test của Bài 2–3 sang dùng Page Object.
- Tiêu chí bắt buộc: mở file test, bấm `Ctrl+F` gõ `By` → **KHÔNG được có kết quả nào**.
- Tiêu chí bắt buộc: **không có Assert** bên trong Page Object; **không có Page Object nào trả về** `IWebElement`.

- Chạy test trước và sau refactor — kết quả phải **giống hệt nhau** (refactor không đổi hành vi). Nộp ảnh cả 2 lần chạy.

Phần D — Áp dụng cho bài Bài 3 (15%)

- Chuyển bài "sắp xếp giá tăng dần" của Bài 3 sang dùng POM.
- Sử dụng nối chuỗi lời gọi (chaining), ví dụ: `loginPage.DangNhap(...).SapXep("lohi").LayDanhSachGia()`
- Gợi ý Assert: NUnit có sẵn `Is.Ordered` để kiểm tra danh sách đã sắp xếp tăng dần.

Phần E — Thẩm định AI (10%)

- Nhờ AI refactor 1 file test sang POM, prompt phải nêu rõ quy ước (BasePage, locator private, không Assert trong Page).
- Ghi lại prompt và kết quả AI trả về.
- CHỈ RA ít nhất **1 chỗ AI làm sai** (hoặc chưa đúng quy ước) và giải thích vì sao sai. Nếu AI làm đúng hết, nêu rõ bạn đã kiểm những tiêu chí nào.

3. Hướng dẫn thực hiện sơ bộ

1. **Refactor từng bước nhỏ**: sau mỗi bước chạy `dotnet build` và `dotnet test`. Đừng sửa một loạt rồi mới build — sẽ có cả chục lỗi cùng lúc.
2. **Bắt đầu từ BasePage**, vì các Page class đều phụ thuộc nó. Nhớ `using OpenQA.Selenium.Support.UI;` cho WebDriverWait.
3. **Chuyển locator trước, hành vi sau**: copy các `By` từ file test sang Page class, đổi thành `private readonly`.
4. Trong Page class dùng `Tim(locator)` thay cho `driver.FindElement(locator)` — để tự động có Explicit Wait của Bài 3.
5. **Quy tắc trả về**: hành vi chuyển trang → `return new TrangMoi(driver);`; hành vi ở lại trang → `return this;`
6. **Namespace**: file trong `Pages/` dùng `namespace FirstTests.Pages;`; file test cần `using FirstTests.Pages;`.
7. **Nếu dùng LINQ** (Select, ToList) mà IDE báo lỗi, thêm `using System.Linq;`.
8. **Tự chấm trước khi nộp** bằng 3 phép thử: không có "By" trong test · không có Assert trong Page · không Page nào trả về IWebElement.
9. **Commit** dùng loại `refactor` theo chuẩn Conventional Commits, ví dụ: `refactor: chuyen LoginTests sang Page Object Model`.

4. Sản phẩm nộp

- Link repository GitHub (**public**) — repo của Bài 1–3.
- Thư mục **`Pages/`**: **`BasePage.cs`**, **`LoginPage.cs`**, **`InventoryPage.cs`**.
- Thư mục **`Tests/`**: các file test đã refactor, không còn chữ "By".
- Thư mục **`screenshots/`**: ảnh kết quả test TRƯỚC và SAU refactor.
- File **`README.md`**: phần ghi nhận & thẩm định AI (Phần E).

5. Tiêu chí chấm

Tiêu chí	Điểm	Ghi chú
Phần A — Cấu trúc Pages/Tests + BasePage đúng chuẩn	**20%**	Có driver, wait, hàm Tim
Phần B — Page class: locator private, hành vi public, trả về Page Object	**25%**	Tên theo hành vi người dùng
Phần C — Refactor test: không còn "By", hành vi không đổi	**30%**	Ảnh test trước & sau
Phần D — Bài sắp xếp giá dùng POM + chaining	**15%**	Gợi ý: Is.Ordered
Phần E — Thẩm định AI: chỉ ra được chỗ AI làm sai	**10%**	Prompt + phân tích
Tổng cộng	**100%**	

Lưu ý: refactor nghĩa là đổi CẤU TRÚC nhưng GIỮ NGUYÊN hành vi. Nếu sau refactor mà test rớt — hoặc tệ hơn là test xanh nhưng không còn kiểm gì — thì bài chưa đạt.